

Лабораторная работа № 2

Методика диагностики Linux-сервера

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - сервер web01;
 - клиент client01;
 - внутренняя сеть VirtualBox.
-

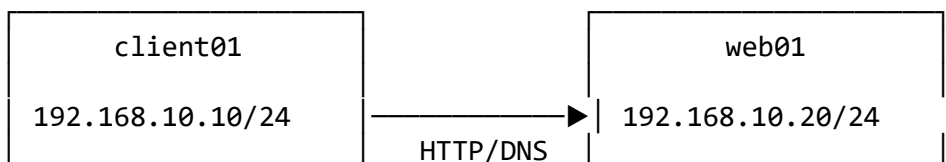
1. Цель работы

Научиться диагностировать неисправности Linux-сервера как последовательный и проверяемый процесс.

В ходе работы необходимо:

- зафиксировать baseline исправного сервера;
 - отличать симптом от причины;
 - проверять ресурсы сервера;
 - диагностировать службу, порт и приложение;
 - определять ошибочный bind address;
 - проверять DNS и системное разрешение имён;
 - проверять локальный firewall;
 - создавать резервную копию конфигурации;
 - проверять синтаксис до применения изменений;
 - подтверждать исправление исходным пользовательским тестом.
-

2. Исходная схема



curl, dig, ping

Nginx, DNS, SSH

Адресация

Устройство	IP-адрес	Маска
client01	192.168.10.10	255.255.255.0
web01	192.168.10.20	255.255.255.0

Имя сервера:

web01.company.test

DNS-запись:

web01.company.test → 192.168.10.20

3. Подготовка стенда

На web01 должны быть установлены:

```
sudo apt update
sudo apt install -y nginx curl dnsutils lsof ufw stress-ng
```

На client01:

```
sudo apt update
sudo apt install -y curl dnsutils
```

В исходном стенде:

- Nginx запущен;
- TCP-порт 80 доступен;
- имя web01.company.test разрешается;
- клиент открывает сайт по IP и имени;
- firewall не блокирует HTTP;
- DNS-служба уже настроена преподавателем.

Перед выполнением Break/Fix-сценариев рекомендуется создать snapshot виртуальной машины web01.

4. Формат диагностики

Каждая неисправность проверяется по одной схеме:

Симптом
↓
Гипотеза
↓
Команда проверки
↓
Фактический вывод
↓
Подтверждённая причина
↓
Исправление
↓
Повтор исходного теста

Нельзя считать причиной первое событие, найденное в журнале. Причина должна подтверждаться командами и фактическим выводом.

5. Базовые задания

Задание 1. Проверить исправное состояние стенда

На web01 смотрим имя сервера и состояние системы:

```
hostnamectl  
uname -r  
uptime
```

Проверяем адреса и маршруты:

```
ip -br link  
ip -br addr  
ip route
```

Проверяем службу Nginx:

```
systemctl is-active nginx  
systemctl status nginx --no-pager
```

Проверяем порт:

```
sudo ss -lntp | grep ':80'
```

Ожидаемый фрагмент:

```
LISTEN 0 511 0.0.0.0:80
```

Проверяем приложение локально:

```
curl -I http://127.0.0.1/  
curl -I http://192.168.10.20/
```

Ожидаемый ответ:

HTTP/1.1 200 OK

На client01 выполняем:

```
ping -c 4 192.168.10.20
curl -I http://192.168.10.20/
curl -I http://web01.company.test/
```

Ожидаемый результат

Сайт доступен:

- локально на сервере;
- с клиента по IP;
- с клиента по имени.

Это исходный пользовательский сценарий. После каждого исправления его необходимо повторять.

Задание 2. Создать baseline исправного сервера

На web01 выполняем:

```
sudo sh -c '
{
    echo "===== DATE ====="
    date

    echo "===== HOST ====="
    hostnamectl
    uname -r
    uptime

    echo "===== CPU ====="
    nproc

    echo "===== MEMORY ====="
    free -h

    echo "===== FILESYSTEMS ====="
    df -hT
    df -i
    lsblk -f
    findmnt

    echo "===== NETWORK ====="
    ip -br link
    ip -br addr
```

```
ip route

echo "===== PORTS ====="
ss -lntup

echo "===== FAILED SERVICES ====="
systemctl --failed

echo "===== NGINX ====="
systemctl status nginx --no-pager
} > /root/baseline-$(date +%F-%H%M).txt
,
```

Находим созданный файл:

```
sudo ls -lh /root/baseline-*
```

Просматриваем его:

```
sudo less /root/baseline-<дата-и-время>.txt
```

Baseline должен содержать:

- дату;
- hostname;
- версию ядра;
- uptime;
- количество CPU;
- состояние RAM;
- файловые системы;
- inode;
- адреса;
- маршруты;
- слушающие порты;
- неисправные службы.

Baseline фиксирует нормальное состояние сервера на конкретную дату. Он используется как точка сравнения, а не как вечный норматив.

Задание 3. Проверить ресурсы сервера

Смотрим количество логических CPU:

```
nproc
```

Смотрим Load Average:

```
uptime
```

Проверяем CPU и процессы:

```
top
```

Для выхода:

```
q
```

Проверяем показатели в динамике:

```
vmstat 1 5
```

Обращаем внимание на поля:

Поле	Значение
r	Процессы, ожидающие CPU
b	Процессы в непрерываемом ожидании
us	Пользовательская нагрузка CPU
sy	Работа ядра
id	Простой CPU
wa	Ожидание ввода-вывода
si	Чтение страниц из Swap
so	Запись страниц в Swap

Проверяем память:

```
free -h
```

Наиболее важное поле:

```
available
```

Смотрим процессы с максимальной нагрузкой:

```
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head
```

Проверяем события OOM:

```
sudo journalctl -k -b |
  grep -i -E 'oom|out of memory|killed process'
```

Вывод

Высокий Load Average сам по себе не доказывает перегрузку CPU. Его нужно сравнивать с:

- количеством CPU;
- %id;
- %wa;
- процессами;

- baseline.
-

Задание 4. Диагностировать ошибку конфигурации Nginx

Сначала создаём резервную копию:

```
sudo cp -a \  
  /etc/nginx/sites-available/default \  
  /etc/nginx/sites-available/default.bak
```

Добавляем ошибочную директиву:

```
echo 'broken_directive on;' |  
  sudo tee -a /etc/nginx/sites-available/default
```

Пытаемся перезапустить службу:

```
sudo systemctl restart nginx
```

На client01 проверяем сайт:

```
curl -I http://192.168.10.20/
```

Симптом

Сайт недоступен.

Проверка службы

На web01:

```
systemctl status nginx --no-pager  
systemctl is-active nginx
```

Ожидаемое состояние:

failed

Проверяем синтаксис:

```
sudo nginx -t
```

Пример вывода:

```
unknown directive "broken_directive"  
nginx: configuration file /etc/nginx/nginx.conf test failed
```

Проверяем журнал:

```
sudo journalctl -u nginx -b --since '-10 min'
```

Причина

Nginx не запускается из-за синтаксической ошибки конфигурации.

Восстанавливаем файл:

```
sudo cp -a \  
  /etc/nginx/sites-available/default.bak \  
  /etc/nginx/sites-available/default
```

Проверяем синтаксис до запуска:

```
sudo nginx -t
```

Ожидаемый результат:

```
syntax is ok  
test is successful
```

Запускаем службу:

```
sudo systemctl restart nginx
```

Повторяем исходный тест:

```
curl -I http://127.0.0.1/  
curl -I http://192.168.10.20/
```

На client01:

```
curl -I http://192.168.10.20/  
curl -I http://web01.company.test/
```

Задание 5. Диагностировать неправильный Bind Address

Создаём новую резервную копию:

```
sudo cp -a \  
  /etc/nginx/sites-available/default \  
  /etc/nginx/sites-available/default.bind-backup
```

Открываем конфигурацию:

```
sudoedit /etc/nginx/sites-available/default
```

Находим строку:

```
listen 80 default_server;
```

Заменяем её:

```
listen 127.0.0.1:80 default_server;
```

Строку IPv6 временно комментируем:

```
# listen [::]:80 default_server;
```

Проверяем:


```
sudo nginx -t  
sudo systemctl reload nginx
```

На web01:

```
curl -I http://127.0.0.1/
```

Ожидается:

HTTP/1.1 200 OK

На client01:

```
curl -I http://192.168.10.20/
```

Ожидается ошибка подключения.

Проверяем порт:

```
sudo ss -lntp | grep ':80'
```

Ожидаемый фрагмент:

127.0.0.1:80

Причина

Nginx принимает соединения только с самого сервера.

Восстанавливаем конфигурацию:

```
sudo cp -a \  
  /etc/nginx/sites-available/default.backup \  
  /etc/nginx/sites-available/default
```

Проверяем и применяем:

```
sudo nginx -t  
sudo systemctl reload nginx
```

Проверяем:

```
sudo ss -lntp | grep ':80'
```

Ожидается:

0.0.0.0:80

На клиенте:

```
curl -I http://192.168.10.20/
```

Задание 6. Диагностировать блокировку Firewall

Работу выполняем через консоль VirtualBox, чтобы не потерять удалённый доступ.

Сначала разрешаем SSH:

```
sudo ufw allow 22/tcp
```

Включаем UFW:

```
sudo ufw --force enable
```

Создаём блокирующее правило:

```
sudo ufw deny 80/tcp
```

Проверяем правила:

```
sudo ufw status verbose
```

На web01 локальная проверка должна работать:

```
curl -I http://127.0.0.1/
```

На client01:

```
curl -I http://192.168.10.20/
```

Симптом

Локально сайт работает, а с клиента соединение не устанавливается.

Проверка

Служба:

```
systemctl is-active nginx
```

Порт:

```
sudo ss -lntp | grep ':80'
```

Firewall:

```
sudo ufw status numbered
```

Причина

Входящие подключения TCP 80 блокируются UFW.

Удаляем правило:

```
sudo ufw delete deny 80/tcp
```

Разрешаем HTTP:

```
sudo ufw allow 80/tcp
```

Проверяем:

```
sudo ufw status verbose
```

Повторяем пользовательский тест:

```
curl -I http://192.168.10.20/
```

Задание 7. Диагностировать проблему разрешения имени

На client01 сначала проверяем исправное состояние:

```
getent hosts web01.company.test  
dig web01.company.test  
curl -I http://web01.company.test/
```

Создаём резервную копию:

```
sudo cp -a /etc/hosts /etc/hosts.backup
```

Добавляем ошибочную локальную запись:

```
echo '192.168.10.99 web01.company.test' |  
sudo tee -a /etc/hosts
```

Проверяем сайт по IP:

```
curl -I http://192.168.10.20/
```

Он должен работать.

Проверяем по имени:

```
curl -I http://web01.company.test/
```

Проверяем системное разрешение имени:

```
getent hosts web01.company.test
```

Ожидаемый неправильный адрес:

```
192.168.10.99
```

Проверяем DNS напрямую:

```
dig web01.company.test
```

DNS может возвращать правильный адрес:

```
192.168.10.20
```

Причина

Системный механизм NSS сначала использует `/etc/hosts`, поэтому ошибочная локальная запись перекрывает правильный DNS-ответ.

Восстанавливаем файл:

```
sudo cp -a /etc/hosts.backup /etc/hosts
```

Проверяем:

```
getent hosts web01.company.test  
curl -I http://web01.company.test/
```

Вывод

Проверки решают разные задачи:

Команда	Что проверяет
<code>getent hosts</code>	Системное разрешение через NSS
<code>dig</code>	Прямой DNS-запрос
<code>curl</code>	Реальный пользовательский HTTP-сценарий

Задание 8. Проверить диски, файловые системы и журналы

На `web01` проверяем свободное место:

```
df -hT
```

Проверяем inode:

```
df -i
```

Проверяем карту устройств:

```
lsblk -f
```

Проверяем точки монтирования:

```
findmnt
```

Проверяем, доступна ли корневая файловая система для записи:

```
findmnt -no SOURCE,FSTYPE,OPTIONS /
```

В списке параметров должно присутствовать:

```
rw
```

Смотрим объём верхних каталогов `/var`:

```
sudo du -xhd1 /var
```

Ищем крупные файлы:

```
sudo find /var -xdev -type f -size +100M -ls
```

Проверяем предупреждения ядра:

```
sudo journalctl -k -b -p warning
```

Проверяем признаки ошибок диска и файловой системы:

```
sudo journalctl -k -b |  
  grep -i -E 'error|fail|i/o|ext4|xfs|nvme'
```

Вывод

Свободное место в `df` не доказывает, что файловая система исправна и доступна для записи. Дополнительно проверяются:

- inode;
 - параметры монтирования;
 - журнал ядра;
 - ошибки накопителя;
 - открытые удалённые файлы.
-

Задание 9. Исследовать контролируемую нагрузку CPU

Запускаем один процесс нагрузки на 60 секунд:

```
stress-ng --cpu 1 --timeout 60s --metrics-brief &
```

Сразу проверяем:

```
uptime
```

Наблюдаем CPU:

```
top
```

В другом терминале:

```
vmstat 1 10
```

Находим процесс:

```
ps aux --sort=-%cpu | head
```

После завершения нагрузки повторяем:

```
uptime
```

```
top
```

Что нужно определить

- вырос ли Load Average;
- уменьшилось ли значение %id;
- какой процесс использовал CPU;
- появились ли признаки I/O Wait;
- вернулась ли система к обычному состоянию.

Вывод

Перед завершением процесса сначала нужно определить:

- его PID;
- владельца;
- назначение;
- связь с systemd-службой.

Службу обычно останавливают через `systemctl`, а обычному процессу сначала отправляют `SIGTERM`, а не `SIGKILL`.

Задание 10. Выполнить итоговый Break/Fix-сценарий

Преподаватель создаёт на `web01` одну неисправность:

- синтаксическая ошибка Nginx;
- неправильный Bind Address;
- блокировка TCP 80 в UFW;
- неправильное разрешение имени;
- остановленная служба Nginx;
- отсутствующий IP-адрес;
- отсутствующий маршрут;
- заполнение тестового каталога;
- ошибка прав доступа к веб-каталогу.

Точный вид неисправности заранее не сообщается.

Диагностику выполняем по этапам.

1. Формулируем симптом

Пример:

С `client01` не открывается `http://web01.company.test`.

2. Повторяем пользовательский сценарий

```
curl -v http://web01.company.test/
```

3. Проверяем разрешение имени

```
getent hosts web01.company.test  
dig web01.company.test
```

4. Проверяем IP-связность

```
ping -c 4 192.168.10.20  
ip route get 192.168.10.20
```

5. Проверяем службу

```
systemctl status nginx --no-pager
```

6. Проверяем порт

```
sudo ss -lntp | grep ':80'
```

7. Проверяем приложение локально

```
curl -v http://127.0.0.1/  
curl -v http://192.168.10.20/
```

8. Проверяем конфигурацию и журнал

```
sudo nginx -t  
sudo journalctl -u nginx -b --since '-30 min'
```

9. Проверяем firewall

```
sudo ufw status verbose  
sudo nft list ruleset
```

10. Перед изменением создаём backup

```
sudo cp -a \  
    /etc/nginx/sites-available/default \  
    /etc/nginx/sites-available/default.bak-$(date +%F-%H%M)
```

11. Исправляем подтверждённую причину

Изменяется только параметр, связь которого с неисправностью доказана.

12. Проверяем синтаксис

```
sudo nginx -t
```

13. Применяем конфигурацию

```
sudo systemctl reload nginx
```

Если служба остановлена:

```
sudo systemctl restart nginx
```

14. Повторяем исходный тест

```
curl -v http://web01.company.test/
```

Исправление считается завершённым только после успешного повторения исходного пользовательского сценария.

6. Дополнительные задания

Дополнительное задание 1. Найти открытый удалённый файл

Создаём файл размером 200 МБ:

```
dd if=/dev/zero \  
  of=/tmp/open-deleted.log \  
  bs=1M count=200 status=progress
```

Открываем его процессом:

```
tail -f /tmp/open-deleted.log >/dev/null &
```

Сохраняем PID:

```
TAIL_PID=$!  
echo "$TAIL_PID"
```

Удаляем имя файла:

```
rm /tmp/open-deleted.log
```

Проверяем:

```
sudo lsof +L1 | grep open-deleted
```

Также можно использовать:

```
sudo lsof | grep deleted
```

Наблюдение

- `du` не видит удалённый файл;
- `df` продолжает учитывать занятое место;
- процесс всё ещё держит файл открытым.

Завершаем процесс:

```
kill "$TAIL_PID"
```

Проверяем, что место освободилось:

```
df -h /tmp  
sudo lsof +L1 | grep open-deleted
```

Дополнительное задание 2. Диагностировать остановку DNS-службы

На web01 определяем DNS-службу:

```
systemctl status dnsmasq --no-pager
```

Останавливаем её:

```
sudo systemctl stop dnsmasq
```

На client01 проверяем:

```
dig @192.168.10.20 web01.company.test
curl -I http://192.168.10.20/
curl -I http://web01.company.test/
```

Ожидаемый результат

- сайт по IP работает;
- прямой DNS-запрос не работает;
- сайт по имени не открывается.

На web01 проверяем:

```
systemctl status dnsmasq --no-pager
sudo ss -lnup | grep ':53'
sudo journalctl -u dnsmasq -b --since '-10 min'
```

Запускаем службу:

```
sudo systemctl start dnsmasq
```

Повторяем тест:

```
dig @192.168.10.20 web01.company.test
curl -I http://web01.company.test/
```

Дополнительное задание 3. Безопасно изменить конфигурацию SSH

Работу выполняем при наличии консоли VirtualBox или второй SSH-сессии.

Создаём резервную копию:

```
sudo cp -a \
/etc/ssh/sshd_config \
/etc/ssh/sshd_config.bak-$(date +%F-%H%M)
```

Открываем конфигурацию:

```
sudoedit /etc/ssh/sshd_config
```

Добавляем безопасный параметр:

```
ClientAliveInterval 60
```

Проверяем синтаксис:

```
sudo sshd -t
```

Если ошибок нет, применяем:

```
sudo systemctl reload ssh
```

Текущую сессию не закрываем.

Открываем вторую SSH-сессию и проверяем вход.

Проверяем состояние:

```
systemctl status ssh --no-pager  
sudo ss -lntp | grep ':22'
```

Вывод

Безопасное изменение SSH включает:

1. консоль или резервную сессию;
 2. backup;
 3. проверку `sshd -t`;
 4. `reload`;
 5. новую тестовую сессию;
 6. готовый rollback.
-

7. Основные команды диагностики

Система и ресурсы

```
date  
hostnamectl  
uptime  
nproc  
free -h  
top  
vmstat 1  
ps aux --sort=-%cpu | head  
ps aux --sort=-%mem | head
```

Диски

```
df -hT  
df -i  
lsblk -f  
findmnt
```

```
sudo du -xhd1 /var
sudo lsof +L1
```

Службы и журналы

```
systemctl status <service>
systemctl is-active <service>
systemctl --failed
journalctl -u <service> -b
journalctl -k -b -p warning
```

Сеть

```
ip -br link
ip -br addr
ip route
ip route get <destination>
ip neigh
ping
```

Порты и приложения

```
sudo ss -lntup
curl -v http://127.0.0.1/
curl -v http://<server-ip>/
```

DNS

```
getent hosts <name>
getent ahosts <name>
dig <name>
dig @<dns-server> <name>
```

Firewall

```
sudo ufw status verbose
sudo nft list ruleset
```

8. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему стенда и таблицу адресации.
3. Имя созданного baseline-файла.
4. Результаты проверки CPU, RAM, диска и inode.
5. Один полностью оформленный Break/Fix-сценарий.
6. Причину неисправности и команды, которыми она подтверждена.
7. Созданную резервную копию конфигурации.

8. Результат синтаксической проверки.
9. Результат повторного пользовательского теста.
10. Краткий вывод по работе.

Шаблон Break/Fix-отчёта

Этап	Результат
Симптом	
Ожидаемое поведение	
Фактическое поведение	
Первая гипотеза	
Команда проверки	
Ключевой вывод	
Подтверждённая причина	
Васкуп	
Исправление	
Проверка результата	
Rollback	
Профилактика	

Не требуется добавлять полный вывод всех команд. В отчёт включаются только строки, подтверждающие причину и результат исправления.

9. Чек-лист выполнения

- ☐ Исправное состояние сайта проверено по IP и имени.
- ☐ Создан baseline сервера.
- ☐ Проверены CPU, RAM, Load Average и Swap.
- ☐ Проверены место, inode и точки монтирования.
- ☐ Диагностирована синтаксическая ошибка Nginx.
- ☐ Проверены служба, порт и приложение.
- ☐ Диагностирован неправильный Bind Address.
- ☐ Проверена блокировка TCP 80 в firewall.
- ☐ Проверены getent и dig.
- ☐ Перед изменениями создавались резервные копии.
- ☐ Конфигурация проверялась до применения.
- ☐ После исправления повторялся исходный пользовательский тест.
- ☐ Итоговый Break/Fix-сценарий оформлен в отчёте.
- ☐ Rollback описан и может быть выполнен.

10. Контрольные вопросы

1. Чем симптом отличается от причины и сопутствующего события?
2. Для чего создаётся baseline исправного сервера?
3. Почему высокий Load Average не всегда означает перегрузку CPU?
4. Почему состояние active (running) не доказывает исправность приложения?
5. Как отличить неисправность службы от неправильного Bind Address?
6. Чем проверки `getent hosts` и `dig` отличаются друг от друга?
7. Почему перед изменением конфигурации нужны backup и проверка синтаксиса?
8. Когда исправление неисправности можно считать подтверждённым?